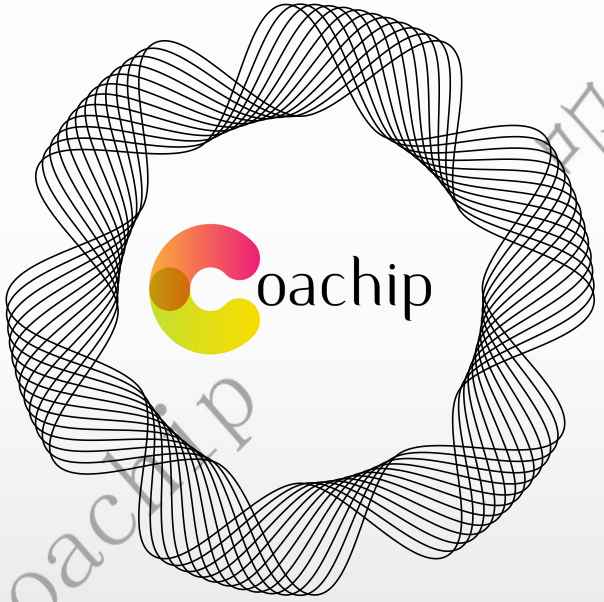


SV随机化约束的概率

叩特电子Coachip

课程内容及目标

课程内容：

- 没有约束的类
- 关系操作
- 关系操作和双向约束
- solve… before 约束引导概率分布



叩特电子Coachip

没有约束的类

```
class unconstrained;
    rand bit x;        // 0或1
    rand bit [1:0] y;  //0,1,2或3
endclass
```

解	x	y	概率
A	0	0	1/8
B	0	1	1/8
C	0	2	1/8
D	0	3	1/8
E	1	0	1/8
F	1	1	1/8
G	1	2	1/8
H	1	3	1/8

叩特电子Coachip

关系操作

```
class impact1;
    rand bit x;  // 0或1
    rand bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      (x==0)-> y==0;
    }
endclass
```

解	x	y	概率
A	0	0	1/5
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	1/5
F	1	1	1/5
G	1	2	1/5
H	1	3	1/5

y的值依赖于x的值

叩特电子Coachip

关系操作

```
class impact1;
    randc bit x;  // 0或1
    rand bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      (x==0)-> y==0;
    }
endclass
```

解	x	y	概率
A	0	0	1/2
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	1/8
F	1	1	1/8
G	1	2	1/8
H	1	3	1/8

y的值依赖于x的值

叩特电子Coachip

关系操作

```
class impact1;
    rand bit x;  // 0或1
    randc bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      (x==0)-> y==0;
    }
endclass
```

解	x	y	概率
A	0	0	1/8
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	1/8
F	1	1	1/4
G	1	2	1/4
H	1	3	1/4

y的值依赖于x的值

叩特电子Coachip

关系操作

```
class impact1;
   randc bit x;  // 0或1
   randc bit [1:0] y;  //0,1,2或3

   constraint c_xy{
      (x==0)-> y==0;
   }
endclass
```

解	x	y	概率
A	0	0	1/4
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	0
F	1	1	1/4
G	1	2	1/4
H	1	3	1/4

y的值依赖于x的值

x=1,y=0是允许的，
但为什么这里为0？
因为randc是轮询的。
x=0, y只能选0；
x=1, y只能选1/2/3里面的。
x=0, y 只能选0，
x=1, y只能选1/2/3里面的

即特电子Coachip

关系操作和双向约束

x对y有约束， y=0时，对x没有约束，如果引入双向约束，则：

```
class impact2;
    rand bit x;  // 0或1
    rand bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      y>0;
     (x==0)-> y==0;
    }
endclass
```

解	x	y	概率
A	0	0	0
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	0
F	1	1	1/3
G	1	2	1/3
H	1	3	1/3

叩特电子Coachip

Solve…before约束引导概率分布

```
class solvebefore;
    rand bit x;  // 0或1
    rand bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      (x==0)-> y==0;
      solve x before y;
    }
endclass
```

解	x	y	概率
A	0	0	1/2
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	1/8
F	1	1	1/8
G	1	2	1/8
H	1	3	1/8

叩特电子Coachip

Solve…before约束引导概率分布

```
class solvebefore;
    rand bit x;  // 0或1
    rand bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      (x==0)-> y==0;
      solve y before x;
    }
endclass
```

解	x	y	概率
A	0	0	1/8
B	0	1	0
C	0	2	0
D	0	3	0
E	1	0	1/8
F	1	1	1/4
G	1	2	1/4
H	1	3	1/4

不要过度使用solve … before，否则会降低计算速度，也会使你的约束难以理解

叩特电子Coachip

练习一

建立一个没有约束的类，打印其随机后的值

```
class unconstrained;
    rand bit x;        // 0或1
    rand bit [1:0] y;  //0,1,2或3
endclass

unconstrained p;
initial begin
    p=new();
    assert(p.randomize());
    $display("x = %d",p.x);
    $display("y = %d",p.y);
end
```

叩特电子Coachip

练习二

建立有依赖关系的约束的类，打印其随机后的值

```
class impact;
    rand bit x;  // 0或1
    rand bit [1:0] y;  //0,1,2或3

    constraint c_xy{
       (x==0)-> y==0;
    }
endclass
```

```
impact p;

initial begin
    p=new();
    assert(p.randomize());
    $display("x = %d",p.x);
    $display("y = %d",p.y);
end
```

叩特电子Coachip

练习三

建立有依赖关系的双向约束的类，打印其随机后的值

```
class impact;
  rand bit x;  // 0或1
  rand bit [1:0] y;  //0,1,2或3

  constraint c_xy{
     y>0;
    (x==0)-> y==0;
  }
endclass
```

```
impact p;

initial begin
    p=new();
    assert(p.randomize());
    $display("x = %d",p.x);
    $display("y = %d",p.y);
end
```

叩特电子Coachip

练习四

建立有**solve … before**引导约束的类，打印其随机后的值

```
class impact;
  rand bit x;  // 0或1
  rand bit [1:0] y;  //0,1,2或3

  constraint c_xy{
    (x==0)-> y==0;
    solve y before x;
  }
endclass
```

```
impact p;

initial begin
  p=new();
  assert(p.randomize());
  $display("x = %d",p.x);
  $display("y = %d",p.y);
end
```

叩特电子Coachip

练习五

体会rand和randc的区别，将y的变量定义rand改为randc，打印其随机后的值

```
class impact;
    rand bit x;  // 0或1
    randc bit [1:0] y;  //0,1,2或3

    constraint c_xy{
      (x==0)-> y==0;
      solve y before x;
    }
endclass
```

```
impact p;

initial begin
    p=new();
    assert(p.randomize());
    $display("x = %d",p.x);
    $display("y = %d",p.y);
end
```

叩特电子Coachip

Q&A

昂特电子Coachip